

第12章 Java内存模型与线程

并发处理的广泛应用是Amdahl定律代替摩尔定律^[1]成为计算机性能发展源动力的根本原因，也是人类压榨计算机运算能力的最有力武器。

[1] Amdahl定律通过系统中并行化与串行化的比重来描述多处理器系统能获得的运算加速能力，摩尔定律则用于描述处理器晶体管数量与运行效率之间的发展关系。这两个定律的更替代表了近年来硬件发展从追求处理器频率到追求多核心并行处理的发展过程。

12.1 概述

多任务处理在现代计算机操作系统中几乎已是一项必备的功能了。在许多场景下，让计算机同时去做几件事情，不仅是因为计算机的运算能力强大了，还有一个很重要的原因是计算机的运算速度与它的存储和通信子系统的速度差距太大，大量的时间都花费在磁盘I/O、网络通信或者数据库访问上。如果不希望处理器在大部分时间里都处于等待其他资源的空闲状态，就必须使用一些手段去把处理器的运算能力“压榨”出来，否则就会造成很大的性能浪费，而让计算机同时处理几项任务则是最容易想到，也被证明是非常有效的“压榨”手段。

除了充分利用计算机处理器的能力外，一个服务端要同时对多个客户端提供服务，则是另一个更具体的并发应用场景。衡量一个服务性能的高低好坏，每秒事务处理数（Transactions Per Second，TPS）是重要的指标之一，它代表着一秒内服务端平均能响应的请求总数，而TPS值与程序的并发能力又有非常密切的关系。对于计算量相同的任务，程序线程并发协调得越有条不紊，效率自然就会越高；反之，线程之间频繁争用数据，互相阻塞甚至死锁，将会大大降低程序的并发能力。

服务端的应用是Java语言最擅长的领域之一，这个领域的应用占了Java应用中最大的一块份额[1]，不过如何写好并发应用程序却又是服务端程序开发的难点之一，处理好并发方面的问题通常需要更多的编码经验来支持。幸好Java语言和虚拟机提供了许多工具，把并发编程的门槛降低了不少。各种中间件服务器、各类框架也都努力地替程序员隐藏尽可能多的线程并发细节，使得程序员在编码时能更关注业务逻辑，而不是花费大部分时间去关注此服务会同时被多少人调用、如何处理数据争用、协调硬件资源。但是无论语言、中间件和框架再如何先进，开发人员都不应期望它们能独立完成所有并发处理的事情，了解并发的内幕仍然是成为一个高级程序员不可缺少的课程。

“高效并发”是本书讲解Java虚拟机的最后一个部分，将会向读者介绍虚拟机如何实现多线程、多线程之间由于共享和竞争数据而导致的一系列问题及解决方案。

[1] 必须以代码的总体规模来衡量，服务端应用不能与JavaCard、移动终端这些领域去比绝对数量。

12.2 硬件的效率与一致性

在正式讲解Java虚拟机并发相关的知识之前，我们先花费一点时间去了解一下物理计算机中的并发问题。物理机遇到的并发问题与虚拟机中的情况有很多相似之处，物理机对并发的处理方案对虚拟机的实现也有相当大的参考意义。

“让计算机并发执行若干个运算任务”与“更充分地利用计算机处理器的效能”之间的因果关系，看起来理所当然，实际上它们之间的关系并没有想象中那么简单，其中一个重要的复杂性的来源是绝大多数的运算任务都不可能只靠处理器“计算”就能完成。处理器至少要与内存交互，如读取运算数据、存储运算结果等，这个I/O操作就是很难消除的（无法仅靠寄存器来完成所有运算任务）。由于计算机的存储设备与处理器的运算速度有着几个数量级的差距，所以现代计算机系统都不得不加入一层或多层读写速度尽可能接近处理器运算速度的高速缓存（Cache）来作为内存与处理器之间的缓冲：将运算需要使用的数据复制到缓存中，让运算能快速进行，当运算结束后再从缓存同步回内存之中，这样处理器就无须等待缓慢的内存读写了。

基于高速缓存的存储交互很好地解决了处理器与内存速度之间的矛盾，但是也为计算机系统带来更高的复杂度，它引入了一个新的问题：缓存一致性（Cache Coherence）。在多路处理器系统中，每个处理器都有自己的高速缓存，而它们又共享同一主内存（Main Memory），这种系统称为共享内存多核系统（Shared Memory Multiprocessors System），如图12-1所示。当多个处理器的运算任务都涉及同一块主内存区域时，将可能导致各自的缓存数据不一致。如果真的发生这种情况，那同步回到主内存时该以谁的缓存数据为准呢？为了解决一致性的问题，需要各个处理器访问缓存时都遵循一些协议，在读写时要根据协议来进行操作，这类协议有MSI、MESI（Illinois Protocol）、MOSI、Synapse、Firefly及Dragon Protocol等。从本章开始，我们将会频繁见到“内存模型”一词，它可以理解为在特定的操作协议下，对特定的内存或高速缓存进行读写访问的过程抽象。不同架构的物理机器可以拥有不一样的内存模型，而Java虚拟机也有自己的内存模型，并且与这里介绍的内存访问操作及硬件的缓存访问操作具有高度的可类比性。

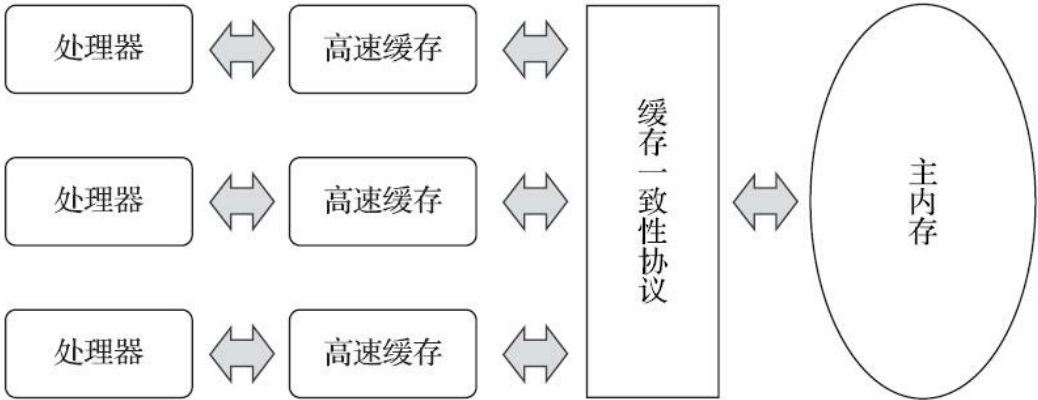


图12-1 处理器、高速缓存、主内存间的交互关系

除了增加高速缓存之外，为了使处理器内部的运算单元能尽量被充分利用，处理器可能会对输入代码进行乱序执行（Out-Of-Order Execution）优化，处理器会在计算之后将乱序执行的结果重组，保证该结果与顺序执行的结果是一致的，但并不保证程序中各个语句计算的先后顺序与输入代码中的顺序一致，因此如果存在一个计算任务依赖另外一个计算任务的中间结果，那么其顺序性并不能靠代码的先后顺序来保证。与处理器的乱序执行优化类似，Java虚拟机的即时编译器中也有指令重排序（Instruction Reorder）优化。

12.3 Java内存模型

《Java虚拟机规范》^[1]中曾试图定义一种“Java内存模型”^[2]（Java Memory Model, JMM）来屏蔽各种硬件和操作系统的内存访问差异，以实现让Java程序在各种平台下都能达到一致的内存访问效果。在此之前，主程序语言（如C和C++等）直接使用物理硬件和操作系统的内存模型。因此，由于不同平台上内存模型的差异，有可能导致程序在一套平台上并发完全正常，而在另外一套平台上并发访问却经常出错，所以在某些场景下必须针对不同的平台来编写程序。

定义Java内存模型并非一件容易的事情，这个模型必须定义得足够严谨，才能让Java的并发内存访问操作不会产生歧义；但是也必须定义得足够宽松，使得虚拟机的实现能有足够的自由空间去利用硬件的各种特性（寄存器、高速缓存和指令集中某些特有的指令）来获取更好的执行速度。经过长时间的验证和修补，直至JDK 5（实现了JSR-133^[3]）发布后，Java内存模型才终于成熟、完善起来了。

[1] 在《Java虚拟机规范》的第2版及之前，专门有一章“Threads and Locks”来描述内存模型，后来由于这部分内容难以把握宽紧限度，被反复修正更新，从第3版（Java SE 7版）开始索性就被移除出规范，独立以JSR形式维护。

[2] 本书中的Java内存模型都特指目前正在使用的，在JDK 1.2之后建立起来并在JDK 5中完善过的内存模型。

[3] JSR-133: Java Memory Model and Thread Specification Revision（Java内存模型和线程规范修订）。

12.3.1 主内存与工作内存

Java内存模型的主要目的是定义程序中各种变量的访问规则，即关注在虚拟机中把变量值存储到内存和从内存中取出变量值这样的底层细节。此处的变量（Variables）与Java编程中所说的变量有所区别，它包括了实例字段、静态字段和构成数组对象的元素，但是不包括局部变量与方法参数，因为后者是线程私有的^[1]，不会被共享，自然就不会存在竞争问题。为了获得更好的执行效能，Java内存模型并没有限制执行引擎使用处理器的特定寄存器或缓存来和主内存进行交互，也没有限制即时编译器是否要进行调整代码执行顺序这类优化措施。

Java内存模型规定了所有的变量都存储在主内存（Main Memory）中（此处的主内存与介绍物理硬件时提到的主内存名字一样，两者也可以类比，但物理上它仅是虚拟机内存的一部分）。每条线程还有自己的工作内存（Working Memory，可与前面讲的处理器高速缓存类比），线程的工作内存中保存了被该线程使用的变量的主内存副本^[2]，线程对变量的所有操作（读取、赋值等）都必须在工作内存中进行，而不能直接读写主内存中的数据^[3]。不同的线程之间也无法直接访问对方工作内存中的变量，线程间变量值的传递均需要通过主内存来完成，线程、主内存、工作内存三者的交互关系如图12-2所示，注意与图12-1进行对比。

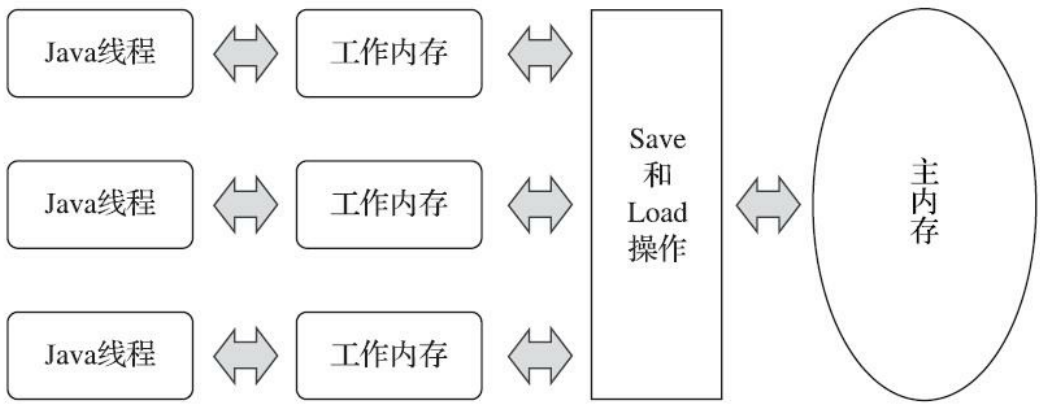


图12-2 线程、主内存、工作内存三者的交互关系（请与图12-1对比）

这里所讲的主内存、工作内存与第2章所讲的Java内存区域中的Java堆、栈、方法区等并不是同一个层次的对内存的划分，这两者基本上是没有关系的。如果两者一定要勉强对应起来，那么从变量、主内存、工作内存的定义来看，主内存主要对应于Java堆中的对象实例数据部分^[4]，而工作内存则对应于虚拟机栈中的部分区域。从更基础的层次上说，主内存直接对应于物理硬件的内存，而为了获取更好的运行速度，虚拟机（或者是硬件、操作系统本身的优化措施）可能会让工作内存优先存储于寄存器和高速缓存中，因为程序运行时主要访问的是工作内存。

[1] 此处请读者注意区分概念：如果局部变量是一个reference类型，它引用的对象在Java堆中可被各个

线程共享，但是reference本身在Java栈的局部变量表中是线程私有的。

[2] 有部分读者会对这段描述中的“副本”提出疑问，如“假设线程中访问一个10MB大小的对象，也会把这10MB的内存复制一份出来吗？”，事实上并不会如此，这个对象的引用、对象中某个在线程访问到的字段是有可能被复制的，但不会有虚拟机把整个对象复制一次。

[3] 根据《Java虚拟机规范》的约定，volatile变量依然有工作内存的拷贝，但是由于它特殊的操作顺序性规定（后文会讲到），所以看起来如同直接在主内存中读写访问一般，因此这里的描述对于volatile也并不存在例外。

[4] 除了实例数据，Java堆还保存了对象的其他信息，对于HotSpot虚拟机来讲，有Mark Word（存储对象哈希码、GC标志、GC年龄、同步锁等信息）、Klass Point（指向存储类型元数据的指针）及一些用于字节对齐补白的填充数据（如果实例数据刚好满足8字节对齐，则可以不存在补白）。

12.3.2 内存间交互操作

关于主内存与工作内存之间具体的交互协议，即一个变量如何从主内存拷贝到工作内存、如何从工作内存同步回主内存这一类的实现细节，Java内存模型中定义了以下8种操作来完成。Java虚拟机实现时必须保证下面提及的每一种操作都是原子的、不可再分的（对于double和long类型的变量来说，load、store、read和write操作在某些平台上允许有例外，这个问题在12.3.4节会专门讨论）^[1]。

- lock（锁定）：作用于主内存的变量，它把一个变量标识为一条线程独占的状态。

- unlock（解锁）：作用于主内存的变量，它把一个处于锁定状态的变量释放出来，释放后的变量才可以被其他线程锁定。

- read（读取）：作用于主内存的变量，它把一个变量的值从主内存传输到线程的工作内存中，以便随后的load动作使用。

- load（载入）：作用于工作内存的变量，它把read操作从主内存中得到的变量值放入工作内存的变量副本中。

- use（使用）：作用于工作内存的变量，它把工作内存中一个变量的值传递给执行引擎，每当虚拟机遇到一个需要使用变量的值的字节码指令时将会执行这个操作。

- assign（赋值）：作用于工作内存的变量，它把一个从执行引擎接收的值赋给工作内存的变量，每当虚拟机遇到一个给变量赋值的字节码指令时执行这个操作。

- store（存储）：作用于工作内存的变量，它把工作内存中一个变量的值传送到主内存中，以便随后的write操作使用。

- write（写入）：作用于主内存的变量，它把store操作从工作内存中得到的变量的值放入主内存的变量中。

如果要把一个变量从主内存拷贝到工作内存，那就要按顺序执行read和load操作，如果要把变量从工作内存同步回主内存，就要按顺序执行store和write操作。注意，Java内存模型只要求上述两个操作必须按顺序执行，但不要求是连续执行。也就是说read与load之间、store与write之间是可插入其他指令的，如对主内存中的变量a、b进行访问时，一种可能出现的顺序是read a、read b、load b、load a。除此之外，Java内存模型还规定了在执行上述8种基本操作时必须满足如下规则：

- 不允许read和load、store和write操作之一单独出现，即不允许一个变量从主内存读取了但工作内存不接受，或者工作内存发起回写了但主内存不接受的情况出现。

- 不允许一个线程丢弃它最近的assign操作，即变量在工作内存中改变了之后必须把该变化同步回主内存。

- 不允许一个线程无原因地（没有发生过任何assign操作）把数据从线程的工作内存同步回主内存中。

- 一个新的变量只能在主内存中“诞生”，不允许在工作内存中直接使用一个未被初始化（load或assign）的变量，换句话说就是对一个变量实施use、store操作之前，必须先执行assign和load操作。

- 一个变量在同一个时刻只允许一条线程对其进行lock操作，但lock操作可以被同一条线程重复执行多次，多次执行lock后，只有执行相同次数的unlock操作，变量才会被解锁。

- 如果对一个变量执行lock操作，那将会清空工作内存中此变量的值，在执行引擎使用这个变量前，需要重新执行load或assign操作以初始化变量的值。

- 如果一个变量事先没有被lock操作锁定，那就不允许对它执行unlock操作，也不允许去unlock一个被其他线程锁定的变量。

- 对一个变量执行unlock操作之前，必须先把此变量同步回主内存中（执行store、write操作）。

这8种内存访问操作以及上述规则限定，再加上稍后会介绍的专门针对volatile的一些特殊规定，就已经能准确地描述出Java程序中哪些内存访问操作在并发下才是安全的。这种定义相当严谨，但也是极为烦琐，实践起来更是无比麻烦。可能部分读者阅读到这里已经对多线程开发产生恐惧感了，后来Java设计团队大概也意识到了这个问题，将Java内存模型的操作简化为read、write、lock和unlock四种，但这只是语言描述上的等价化简，Java内存模型的基础设计并未改变，即使是这四操作种，对于普通用户来说阅读使用起来仍然并不方便。不过读者对此无须过分担忧，除了进行虚拟机开发的团队外，大概没有其他开发人员会以这种方式来思考并发问题，我们只需要理解Java内存模型的定义即可。12.3.6节将介绍这种定义的一个等效判断原则——先行发生原则，用来确定一个操作在并发环境下是否安全的。

[1] 基于理解难度和严谨性考虑，最新的JSR-133文档中，已经放弃了采用这8种操作去定义Java内存模型的访问协议，缩减为4种（仅是描述方式改变了，Java内存模型并没有改变）。

12.3.3 对于volatile型变量的特殊规则

关键字volatile可以说是Java虚拟机提供的最轻量级的同步机制，但是它并不容易被正确、完整地理解，以至于许多程序员都习惯去避免使用它，遇到需要处理多线程数据竞争问题的时候一律使用synchronized来进行同步。了解volatile变量的语义对后面理解多线程操作的其他特性很有意义，在本节中我们将多花费一些篇幅介绍volatile到底意味着什么。

Java内存模型为volatile专门定义了一些特殊的访问规则，在介绍这些比较拗口的规则定义之前，先用一些不那么正式，但通俗易懂的语言来介绍一下这个关键字的作用。

当一个变量被定义成volatile之后，它将具备两项特性：第一项是保证此变量对所有线程的可见性，这里的“可见性”是指当一条线程修改了这个变量的值，新值对于其他线程来说是可以立即得知的。而普通变量并不能做到这一点，普通变量的值在线程间传递时均需要通过主内存来完成。比如，线程A修改一个普通变量的值，然后向主内存进行回写，另外一条线程B在线程A回写完成了之后再对主内存进行读取操作，新变量值才会对线程B可见。

关于volatile变量的可见性，经常会被开发人员误解，他们会误以为下面的描述是正确的：“volatile变量对所有线程是立即可见的，对volatile变量所有的写操作都能立刻反映到其他线程之中。换句话说，volatile变量在各个线程中是一致的，所以基于volatile变量的运算在并发下是线程安全的”。这句话的论据部分并没有错，但是由其论据并不能得出“基于volatile变量的运算在并发下是线程安全的”这样的结论。volatile变量在各个线程的工作内存中是不存在一致性问题的（从物理存储的角度看，各个线程的工作内存中volatile变量也可以存在不一致的情况，但由于每次使用之前都要先刷新，执行引擎看不到不一致的情况，因此可以认为不存在一致性问题），但是Java里面的运算操作符并非原子操作，这导致volatile变量的运算在并发下一样是不安全的，我们可以通过一段简单的演示来说明原因，请看代码清单12-1中演示的例子。

代码清单12-1 volatile的运算^[1]

```
/**
 * volatile变量自增运算测试
 *
 * @author zzm
 */
public class VolatileTest {

    public static volatile int race = 0;

    public static void increase() {
        race++;
    }

    private static final int THREADS_COUNT = 20;

    public static void main(String[] args) {
        Thread[] threads = new Thread[THREADS_COUNT];
        for (int i = 0; i < THREADS_COUNT; i++) {
            threads[i] = new Thread(new Runnable() {
                @Override
                public void run() {
                    for (int i = 0; i < 10000; i++) {
                        increase();
                    }
                }
            })
        }
    }
}
```

```

    }
    });
    threads[i].start();
}

// 等待所有累加线程都结束
while (Thread.activeCount() > 1)
    Thread.yield();

System.out.println(race);
}
}

```

这段代码发起了20个线程，每个线程对`race`变量进行10000次自增操作，如果这段代码能够正确并发的话，最后输出的结果应该是200000。读者运行完这段代码之后，并不会获得期望的结果，而且会发现每次运行程序，输出的结果都不一样，都是一个小于200000的数字。这是为什么呢？

问题就出在自增运算“`race++`”之中，我们用Javap反编译这段代码后会得到代码清单12-2所示，发现只有一行代码的`increase()`方法在Class文件中是由4条字节码指令构成（`return`指令不是由`race++`产生的，这条指令可以不计算），从字节码层面上已经很容易分析出并发失败的原因了：当`getstatic`指令把`race`的值取到操作栈顶时，`volatile`关键字保证了`race`的值在此时是正确的，但是在执行`iconst_1`、`iadd`这些指令的时候，其他线程可能已经把`race`的值改变了，而操作栈顶的值就变成了过期的数据，所以`putstatic`指令执行后就可能把较小的`race`值同步回主内存之中。

代码清单12-2 VolatileTest的字节码

```

public static void increase();
Code:
    Stack=2, Locals=0, Args_size=0
    0:  getstatic      #13; //Field race:I
    3:  iconst_1
    4:  iadd
    5:  putstatic      #13; //Field race:I
    8:  return
LineNumberTable:
    line 14: 0
    line 15: 8

```

实事求是地说，笔者使用字节码来分析并发问题仍然是不严谨的，因为即使编译出来只有一条字节码指令，也并不意味执行这条指令就是一个原子操作。一条字节码指令在解释执行时，解释器要运行许多行代码才能实现它的语义。如果是编译执行，一条字节码指令也可能转化成若干条本地机器码指令。此处使用-XX: +PrintAssembly参数输出反汇编来分析才会更加严谨一些，但是考虑到读者阅读的方便性，并且字节码已经能很好地说明问题，所以此处使用字节码来解释。

由于`volatile`变量只能保证可见性，在不符合以下两条规则的运算场景中，我们仍然要通过加锁（使用`synchronized`、`java.util.concurrent`中的锁或原子类）来保证原子性：

- 运算结果并不依赖变量的当前值，或者能够确保只有单一的线程修改变量的值。
- 变量不需要与其他的状态变量共同参与不变约束。

而在像代码清单12-3所示的这类场景中就很适合使用`volatile`变量来控制并发，当`shutdown()`方法被调用时，能保证所有线程中执行的`doWork()`方法都立即停下来。

代码清单12-3 volatile的使用场景

```
volatile boolean shutdownRequested;

public void shutdown() {
    shutdownRequested = true;
}

public void doWork() {
    while (!shutdownRequested) {
        // 代码的业务逻辑
    }
}
```

使用volatile变量的第二个语义是禁止指令重排序优化，普通的变量仅会保证在该方法的执行过程中所有依赖赋值结果的地方都能获取到正确的结果，而不能保证变量赋值操作的顺序与程序代码中的执行顺序一致。因为在同一个线程的方法执行过程中无法感知到这点，这就是Java内存模型中描述的所谓“线程内表现为串行的语义”（Within-Thread As-If-Serial Semantics）。

上面描述仍然比较拗口难明，我们还是继续通过一个例子来看看为何指令重排序会干扰程序的并发执行。演示程序如代码清单12-4所示。

代码清单12-4 指令重排序

```
Map configOptions;
char[] configText;
// 此变量必须定义为volatile
volatile boolean initialized = false;

// 假设以下代码在线程A中执行
// 模拟读取配置信息，当读取完成后
// 将initialized设置为true，通知其他线程配置可用
configOptions = new HashMap();
configText = readConfigFile(fileName);
processConfigOptions(configText, configOptions);
initialized = true;

// 假设以下代码在线程B中执行
// 等待initialized为true，代表线程A已经把配置信息初始化完成
while (!initialized) {
    sleep();
}
// 使用线程A中初始化好的配置信息
doSomethingWithConfig();
```

代码清单12-4中所示的程序是一段伪代码，其中描述的场景是开发中常见配置读取过程，只是我们在处理配置文件时一般不会出现并发，所以没有察觉这会有问题。读者试想一下，如果定义initialized变量时没有使用volatile修饰，就可能会由于指令重排序的优化，导致位于线程A中最后一条代码“initialized=true”被提前执行（这里虽然使用Java作为伪代码，但所指的重排序优化是机器级的优化操作，提前执行是指这条语句对应的汇编代码被提前执行），这样在线程B中使用配置信息的代码就可能出现错误，而volatile关键字则可以避免此类情况的发生^[2]。

指令重排序是并发编程中最容易导致开发人员产生疑惑的地方之一，除了上面伪代码的例子之外，笔者再举一个可以实际操作运行的例子来分析volatile关键字是如何禁止指令重排序优化的。代码

清单12-5所示是一段标准的双锁检测（Double Check Lock, DCL）单例^[3]代码，可以观察加入volatile和未加入volatile关键字时所生成的汇编代码的差别（如何获得即时编译的汇编代码？请参考第4章关于HSDIS插件的介绍）。

代码清单12-5 DCL单例模式

```
public class Singleton {  
  
    private volatile static Singleton instance;  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            synchronized (Singleton.class) {  
                if (instance == null) {  
                    instance = new Singleton();  
                }  
            }  
        }  
        return instance;  
    }  
  
    public static void main(String[] args) {  
        Singleton.getInstance();  
    }  
}
```

编译后，这段代码对instance变量赋值的部分如代码清单12-6所示。

代码清单12-6 对instance变量赋值

```
0x01a3de0f: mov     $0x3375cdb0,%esi      ;...beb0cd75 33  
                                ; {oop('Singleton')}  
0x01a3de14: mov     %eax,0x150(%esi)      ;...89865001 0000  
0x01a3de1a: shr     $0x9,%esi            ;...clee09  
0x01a3de1d: movb    $0x0,0x1104800(%esi) ;...c6860048 100100  
0x01a3de24: lock addl $0x0,(%esp)        ;...f0830424 00  
                                ;*putstatic instance  
                                ; - Singleton::getInstance@24
```

通过对比发现，关键变化在于有volatile修饰的变量，赋值后（前面mov%eax, 0x150(%esi)这句便是赋值操作）多执行了一个“lock addl\$0x0, (%esp)”操作，这个操作的作用相当于一个内存屏障

（Memory Barrier或Memory Fence，指重排序时不能把后面的指令重排序到内存屏障之前的位置，注意不要与第3章中介绍的垃圾收集器用于捕获变量访问的内存屏障互相混淆），只有一个处理器访问内存时，并不需要内存屏障；但如果有两个或更多处理器访问同一块内存，且其中有一个在观测另一个，就需要内存屏障来保证一致性了。

这句指令中的“addl\$0x0, (%esp)”（把ESP寄存器的值加0）显然是一个空操作，之所以用这个空操作而不是空操作专用指令nop，是因为IA32手册规定lock前缀不允许配合nop指令使用。这里的关键在于lock前缀，查询IA32手册可知，它的作用是将本处理器的缓存写入了内存，该写入动作也会引起别的处理器或者别的内核无效化（Invalidate）其缓存，这种操作相当于对缓存中的变量做了一次前面介绍Java内存模式中所说的“store和write”操作^[4]。所以通过这样一个空操作，可让前面volatile变量的修改对其他处理器立即可见。

那为何说它禁止指令重排序呢？从硬件架构上讲，指令重排序是指处理器采用了允许将多条指令不按程序规定的顺序分开发送给各个相应的电路单元进行处理。但并不是说指令任意重排，处理器必须能正确处理指令依赖情况保障程序能得出正确的执行结果。譬如指令1把地址A中的值加10，指令2把地址A中的值乘以2，指令3把地址B中的值减去3，这时指令1和指令2是有依赖的，它们之间的顺序不能重排—— $(A+10)*2$ 与 $A*2+10$ 显然不相等，但指令3可以重排到指令1、2之前或者中间，只要保证处理器执行后面依赖到A、B值的操作时能获取正确的A和B值即可。所以在同一个处理器中，重排序过的代码看起来依然是有序的。因此，`lock addl$0x0, (%esp)`指令把修改同步到内存时，意味着所有之前的操作都已经执行完成，这样便形成了“指令重排序无法越过内存屏障”的效果。

解决了volatile的语义问题，再来看看在众多保障并发安全的工具中选用volatile的意义——它能让我们的代码比使用其他的同步工具更快吗？在某些情况下，volatile的同步机制的性能确实要优于锁（使用synchronized关键字或java.util.concurrent包里面的锁），但是由于虚拟机对锁实行的许多消除和优化，使得我们很难确切地说volatile就会比synchronized快上多少。如果让volatile自己与自己比较，那可以确定一个原则：volatile变量读操作的性能消耗与普通变量几乎没有什么差别，但是写操作则可能会慢上一些，因为它需要在本地代码中插入许多内存屏障指令来保证处理器不发生乱序执行。不过即便如此，大多数场景下volatile的总开销仍然要比锁来得更低。我们在volatile与锁中选择的唯一判断依据仅仅是volatile的语义能否满足使用场景的需求。

本节的最后，我们再回头来看看Java内存模型中对volatile变量定义的特殊规则的定义。假定T表示一个线程，V和W分别表示两个volatile型变量，那么在进行read、load、use、assign、store和write操作时需要满足如下规则：

- 只有当线程T对变量V执行的前一个动作是load的时候，线程T才能对变量V执行use动作；并且，只有当线程T对变量V执行的后一个动作是use的时候，线程T才能对变量V执行load动作。线程T对变量V的use动作可以认为是和线程T对变量V的load、read动作相关联的，必须连续且一起出现。

这条规则要求在工作内存中，每次使用V前都必须先从主内存刷新最新的值，用于保证能看见其他线程对变量V所做的修改。

- 只有当线程T对变量V执行的前一个动作是assign的时候，线程T才能对变量V执行store动作；并且，只有当线程T对变量V执行的后一个动作是store的时候，线程T才能对变量V执行assign动作。线程T对变量V的assign动作可以认为是和线程T对变量V的store、write动作相关联的，必须连续且一起出现。

这条规则要求在工作内存中，每次修改V后都必须立刻同步回主内存中，用于保证其他线程可以看到自己对变量V所做的修改。

- 假定动作A是线程T对变量V实施的use或assign动作，假定动作F是和动作A相关联的load或store动作，假定动作P是和动作F相应的对变量V的read或write动作；与此类似，假定动作B是线程T对变量W实施的use或assign动作，假定动作G是和动作B相关联的load或store动作，假定动作Q是和动作G相应的对变量W的read或write动作。如果A先于B，那么P先于Q。

这条规则要求volatile修饰的变量不会被指令重排序优化，从而保证代码的执行顺序与程序的顺序相同。

- [1] 使用IntelliJ IDEA的读者请注意，在IDEA中运行这段程序，会由于IDE自动创建一条名为Monitor Ctrl-Break的线程（从名字看应该是监控Ctrl-Break中断信号的）而导致while循环无法结束，改为大于2或者用Thread::join()方法代替可以解决该问题。
- [2] volatile屏蔽指令重排序的语义在JDK 5中才被完全修复，此前的JDK中即使将变量声明为volatile也仍然不能完全避免重排序所导致的问题（主要是volatile变量前后的代码仍然存在重排序问题），这一点也是在JDK 5之前的Java中无法安全地使用DCL（双锁检测）来实现单例模式的原因。
- [3] 双重锁定检查是一种在许多语言中都广泛流传的单例构造模式。
- [4] Doug Lea列出了各种处理器架构下的内存屏障指令：<http://gee.cs.oswego.edu/dl/jmm/cookbook.html>。

12.3.4 针对long和double型变量的特殊规则

Java内存模型要求lock、unlock、read、load、assign、use、store、write这八种操作都具有原子性，但是对于64位的数据类型（long和double），在模型中特别定义了一条宽松的规定：允许虚拟机将没有被volatile修饰的64位数据的读写操作划分为两次32位的操作来进行，即允许虚拟机实现自行选择是否要保证64位数据类型的load、store、read和write这四个操作的原子性，这就是所谓的“long和double的非原子性协定”（Non-Atomic Treatment of double and long Variables）。

如果有多个线程共享一个并未声明为volatile的long或double类型的变量，并且同时对它们进行读取和修改操作，那么某些线程可能会读取到一个既不是原值，也不是其他线程修改值的代表了“半个变量”的数值。不过这种读取到“半个变量”的情况是非常罕见的，经过实际测试^[1]，在目前主流平台下商用的64位Java虚拟机中并不会出现非原子性访问行为，但是对于32位的Java虚拟机，譬如比较常用的32位x86平台下的HotSpot虚拟机，对long类型的数据确实存在非原子性访问的风险。从JDK 9起，HotSpot增加了一个实验性的参数-XX: +AlwaysAtomicAccesses（这是JEP 188对Java内存模型更新的一部分内容）来约束虚拟机对所有数据类型进行原子性的访问。而针对double类型，由于现代中央处理器中一般都包含专门用于处理浮点数据的浮点运算器（Floating Point Unit, FPU），用来专门处理单、双精度的浮点数据，所以哪怕是32位虚拟机中通常也不会出现非原子性访问的问题，实际测试也证实了这一点。笔者的看法是，在实际开发中，除非该数据有明确可知的线程竞争，否则我们在编写代码时一般不需要因为这个原因刻意把用到的long和double变量专门声明为volatile。

[1] 根据一篇文章（<https://shipilev.net/blog/2014/all-accesses-are-atomic/>）的介绍，对于这种情况，在ARMv6、ARMv7、x86、x86-AMD64、PowerPC等平台上都进行了实际测试。

12.3.5 原子性、可见性与有序性

介绍完Java内存模型的相关操作和规则后，我们再整体回顾一下这个模型的特征。Java内存模型是围绕着在并发过程中如何处理原子性、可见性和有序性这三个特征来建立的，我们逐个来看一下哪些操作实现了这三个特性。

1. 原子性（Atomicity）

由Java内存模型来直接保证的原子性变量操作包括read、load、assign、use、store和write这六个，我们大致可以认为，基本数据类型的访问、读写都是具备原子性的（例外就是long和double的非原子性协定，读者只要知道这件事情就可以了，无须太过在意这些几乎不会发生的例外情况）。

如果应用场景需要一个更大范围的原子性保证（经常会遇到），Java内存模型还提供了lock和unlock操作来满足这种需求，尽管虚拟机未把lock和unlock操作直接开放给用户使用，但是却提供了更高层次的字节码指令monitorenter和monitorexit来隐式地使用这两个操作。这两个字节码指令反映到Java代码中就是同步块——synchronized关键字，因此在synchronized块之间的操作也具备原子性。

2. 可见性（Visibility）

可见性就是指当一个线程修改了共享变量的值时，其他线程能够立即得知这个修改。上文在讲解volatile变量的时候我们已详细讨论过这一点。Java内存模型是通过在变量修改后将新值同步回主内存，在变量读取前从主内存刷新变量值这种依赖主内存作为传递媒介的方式来实现可见性的，无论是普通变量还是volatile变量都是如此。普通变量与volatile变量的区别是，volatile的特殊规则保证了新值能立即同步到主内存，以及每次使用前立即从主内存刷新。因此我们可以说volatile保证了多线程操作时变量的可见性，而普通变量则不能保证这一点。

除了volatile之外，Java还有两个关键字能实现可见性，它们是synchronized和final。同步块的可见性是由“对一个变量执行unlock操作之前，必须先把此变量同步回主内存中（执行store、write操作）”这条规则获得的。而final关键字的可见性是指：被final修饰的字段在构造器中一旦被初始化完成，并且构造器没有把“this”的引用传递出去（this引用逃逸是一件很危险的事情，其他线程有可能通过这个引用访问到“初始化了一半”的对象），那么在其他线程中就能看见final字段的值。如代码清单12-7所示，变量i与j都具备可见性，它们无须同步就能被其他线程正确访问。

代码清单12-7 final与可见性

```
public static final int i;

public final int j;

static {
    i = 0;
    // 省略后续动作
}

{
    // 也可以选择构造函数中初始化
    j = 0;
    // 省略后续动作
}
```

3.有序性（Ordering）

Java内存模型的有序性在前面讲解volatile时也比较详细地讨论过了，Java程序中天然的有序性可以总结为一句话：如果在本线程内观察，所有的操作都是有序的；如果在一个线程中观察另一个线程，所有的操作都是无序的。前半句是指“线程内似表现为串行的语义”（Within-Thread As-If-Serial Semantics），后半句是指“指令重排序”现象和“工作内存与主内存同步延迟”现象。

Java语言提供了volatile和synchronized两个关键字来保证线程之间操作的有序性，volatile关键字本身就包含了禁止指令重排序的语义，而synchronized则是由“一个变量在同一个时刻只允许一条线程对其进行lock操作”这条规则获得的，这个规则决定了持有同一个锁的两个同步块只能串行地进入。

介绍完并发中三种重要的特性，读者是否发现synchronized关键字在需要这三种特性的时候都可以作为其中一种的解决方案？看起来很“万能”吧？的确，绝大部分并发控制操作都能使用synchronized来完成。synchronized的“万能”也间接造就了它被程序员滥用的局面，越“万能”的并发控制，通常会伴随着越大的性能影响，关于这一点我们将在下一章讲解虚拟机锁优化时再细谈。

12.3.6 先行发生原则

如果Java内存模型中所有的有序性都仅靠volatile和synchronized来完成，那么有很多操作都将会变得非常啰嗦，但是我们在编写Java并发代码的时候并没有察觉到这一点，这是因为Java语言中有一个“先行发生”（Happens-Before）的原则。这个原则非常重要，它是判断数据是否存在竞争，线程是否安全的非常有用的手段。依赖这个原则，我们可以通过几条简单规则一揽子解决并发环境下两个操作之间是否可能存在冲突的所有问题，而不需要陷入Java内存模型苦涩难懂的定义之中。

现在就来看看“先行发生”原则指的是什么。先行发生是Java内存模型中定义的两项操作之间的偏序关系，比如说操作A先行发生于操作B，其实就是在发生操作B之前，操作A产生的影响能被操作B观察到，“影响”包括修改了内存中共享变量的值、发送了消息、调用了方法等。这句话不难理解，但它意味着什么呢？我们可以举个例子来说明一下。如代码清单12-8所示的这三条伪代码。

代码清单12-8 先行发生原则示例1

```
// 以下操作在线程A中执行
i = 1;

// 以下操作在线程B中执行
j = i;

// 以下操作在线程C中执行
i = 2;
```

假设线程A中的操作“i=1”先行发生于线程B的操作“j=i”，那我们就可以确定在线程B的操作执行后，变量j的值一定是等于1，得出这个结论的依据有两个：一是根据先行发生原则，“i=1”的结果可以被观察到；二是线程C还没登场，线程A操作结束之后没有其他线程会修改变量i的值。现在再来考虑线程C，我们依然保持线程A和B之间的先行发生关系，而C出现在线程A和B的操作之间，但是C与B没有先行发生关系，那j的值会是多少呢？答案是不确定！1和2都有可能，因为线程C对变量i的影响可能会被线程B观察到，也可能不会，这时候线程B就存在读取到过期数据的风险，不具备多线程安全性。

下面是Java内存模型下一些“天然的”先行发生关系，这些先行发生关系无须任何同步器协助就已经存在，可以在编码中直接使用。如果两个操作之间的关系不在此列，并且无法从下列规则推导出来，则它们就没有顺序性保障，虚拟机可以对它们随意地进行重排序。

·**程序次序规则**（Program Order Rule）：在一个线程内，按照控制流顺序，书写在前面的操作先行发生于书写在后面的操作。注意，这里说的是控制流顺序而不是程序代码顺序，因为要考虑分支、循环等结构。

·**管程锁定规则**（Monitor Lock Rule）：一个unlock操作先行发生于后面对同一个锁的lock操作。这里必须强调的是“同一个锁”，而“后面”是指时间上的先后。

·**volatile变量规则**（Volatile Variable Rule）：对一个volatile变量的写操作先行发生于后面对这个变量的读操作，这里的“后面”同样是指时间上的先后。

·**线程启动规则**（Thread Start Rule）：Thread对象的start()方法先行发生于此线程的每一个动作。

·**线程终止规则**（Thread Termination Rule）：线程中的所有操作都先行发生于对此线程的终止检测，我们可以通过Thread::join()方法是否结束、Thread::isAlive()的返回值等手段检测线程是否已经终止执行。

·**线程中断规则**（Thread Interruption Rule）：对线程interrupt()方法的调用先行发生于被中断线程的代码检测到中断事件的发生，可以通过Thread::interrupted()方法检测到是否有中断发生。

·**对象终结规则**（Finalizer Rule）：一个对象的初始化完成（构造函数执行结束）先行发生于它的finalize()方法的开始。

·**传递性**（Transitivity）：如果操作A先行发生于操作B，操作B先行发生于操作C，那就可以得出操作A先行发生于操作C的结论。

Java语言无须任何同步手段保障就能成立的先行发生规则有且只有上面这些，下面演示一下如何使用这些规则去判定操作间是否具备顺序性，对于读写共享变量的操作来说，就是线程是否安全。读者还可以从下面这个例子中感受一下“时间上的先后顺序”与“先行发生”之间有什么不同。演示例子如代码清单12-9所示。

代码清单12-9 先行发生原则示例2

```
private int value = 0;

public void setValue(int value){
    this.value = value;
}

public int getValue(){
    return value;
}
```

代码清单12-9中显示的是一组再普通不过的getter/setter方法，假设存在线程A和B，线程A先（时间上的先后）调用了setValue(1)，然后线程B调用了同一个对象的getValue()，那么线程B收到的返回值是什么？

我们依次分析一下先行发生原则中的各项规则。由于两个方法分别由线程A和B调用，不在一个线程中，所以程序次序规则在这里不适用；由于没有同步块，自然就不会发生lock和unlock操作，所以管程锁定规则不适用；由于value变量没有被volatile关键字修饰，所以volatile变量规则不适用；后面的线程启动、终止、中断规则和对对象终结规则也和这里完全没有关系。因为没有有一个适用的先行发生规则，所以最后一条传递性也无从谈起，因此我们可以判定，尽管线程A在操作时间上先于线程B，但是无法确定线程B中getValue()方法的返回结果，换句话说，这里面的操作不是线程安全的。

那怎么修复这个问题呢？我们至少有两种比较简单的方案可以选择：要么把getter/setter方法都定义为synchronized方法，这样就可以套用管程锁定规则；要么把value定义为volatile变量，由于setter方法对value的修改不依赖value的原值，满足volatile关键字使用场景，这样就可以套用volatile变量规则来实现先行发生关系。

通过上面的例子，我们可以得出结论：一个操作“时间上的先发生”不代表这个操作会是“先行发生”。那如果一个操作“先行发生”，是否就能推导出这个操作必定是“时间上的先发生”呢？很遗憾，这个推论也是不成立的。一个典型的例子就是多次提到的“指令重排序”，演示例子如代码清单12-10所示。

代码清单12-10 先行发生原则示例3

```
// 以下操作在同一个线程中执行
int i = 1;
int j = 2;
```

代码清单12-10所示的两条赋值语句在同一个线程之中，根据程序次序规则，“int i=1”的操作先行发生于“int j=2”，但是“int j=2”的代码完全可能先被处理器执行，这并不影响先行发生原则的正确性，因为我们在这条线程之中没有办法感知到这一点。

上面两个例子综合起来证明了一个结论：时间先后顺序与先行发生原则之间基本没有因果关系，所以我们衡量并发安全问题的时候不要受时间顺序的干扰，一切必须以先行发生原则为准。

12.4 Java与线程

并发不一定要依赖多线程（如PHP中很常见的多进程并发），但是在Java里面谈论并发，基本上都与线程脱不开关系。既然本书探讨的是Java虚拟机的特性，那讲到Java线程，我们就从Java线程在虚拟机中的实现开始讲起。

12.4.1 线程的实现

我们知道，线程是比进程更轻量级的调度执行单位，线程的引入，可以把一个进程的资源分配和执行调度分开，各个线程既可以共享进程资源（内存地址、文件I/O等），又可以独立调度。目前线程是Java里面进行处理器资源调度的最基本单位，不过如果日后Loom项目能成功为Java引入纤程（Fiber）的话，可能就会改变这一点。

主流的操作系统都提供了线程实现，Java语言则提供了在不同硬件和操作系统平台下对线程操作的统一处理，每个已经调用过start()方法且还未结束的java.lang.Thread类的实例就代表着一个线程。我们注意到Thread类与大部分的Java类库API有着显著差别，它的所有关键方法都被声明为Native。在Java类库API中，一个Native方法往往就意味着这个方法没有使用或无法使用平台无关的手段来实现（当然也可能是为了执行效率而使用Native方法，不过通常最高效率的手段也就是平台相关的手段）。正因为这个原因，本节的标题被定为“线程的实现”而不是“Java线程的实现”，在稍后介绍的实现方式中，我们也先把Java的技术背景放下，以一个通用的应用程序的角度来看看线程是如何实现的。

实现线程主要有三种方式：使用内核线程实现（1：1实现），使用用户线程实现（1：N实现），使用用户线程加轻量级进程混合实现（N：M实现）。

1. 内核线程实现

使用内核线程实现的方式也被称为1：1实现。内核线程（Kernel-Level Thread，KLT）就是直接由操作系统内核（Kernel，下称内核）支持的线程，这种线程由内核来完成线程切换，内核通过操纵调度器（Scheduler）对线程进行调度，并负责将线程的任务映射到各个处理器上。每个内核线程可以视为内核的一个分身，这样操作系统就有能力同时处理多件事情，支持多线程的内核就称为多线程内核（Multi-Threads Kernel）。

程序一般不会直接使用内核线程，而是使用内核线程的一种高级接口——轻量级进程（Light Weight Process，LWP），轻量级进程就是我们通常意义上所讲的线程，由于每个轻量级进程都由一个内核线程支持，因此只有先支持内核线程，才能有轻量级进程。这种轻量级进程与内核线程之间1：1的关系称为一对一的线程模型，如图12-3所示。

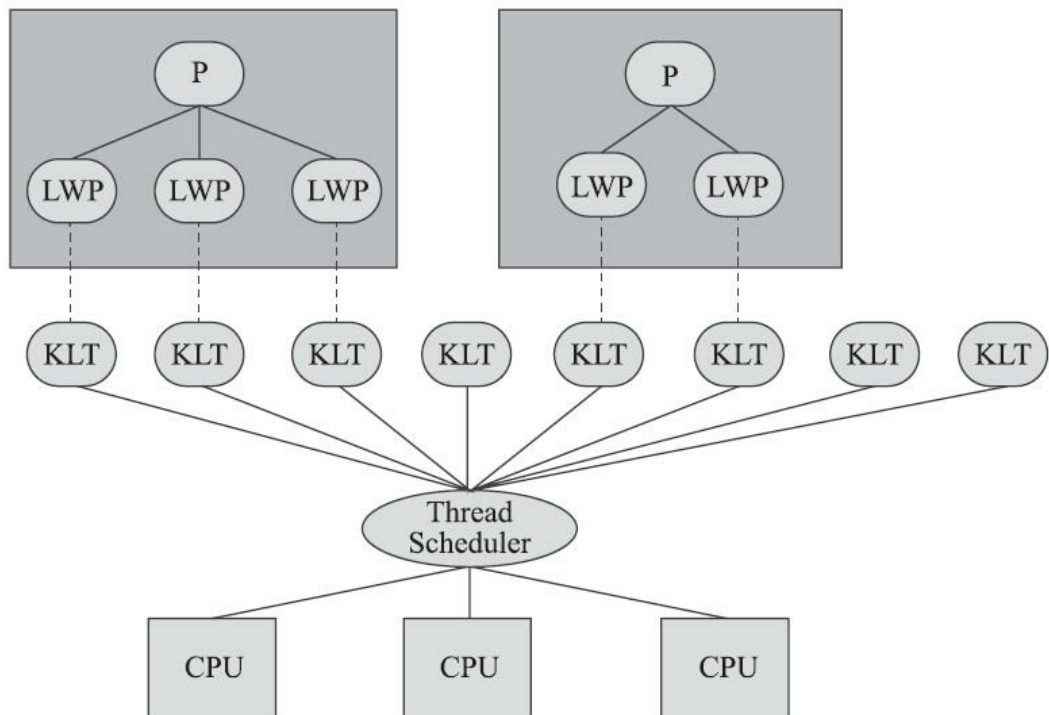


图12-3 轻量级进程与内核线程之间1：1的关系

由于内核线程的支持，每个轻量级进程都成为一个独立的调度单元，即使其中某一个轻量级进程在系统调用中被阻塞了，也不会影响整个进程继续工作。轻量级进程也具有它的局限性：首先，由于是基于内核线程实现的，所以各种线程操作，如创建、析构及同步，都需要进行系统调用。而系统调用的代价相对较高，需要在用户态（User Mode）和内核态（Kernel Mode）中来回切换。其次，每个轻量级进程都需要有一个内核线程的支持，因此轻量级进程要消耗一定的内核资源（如内核线程的栈空间），因此一个系统支持轻量级进程的数量是有限的。

2. 用户线程实现

使用用户线程实现的方式被称为1：N实现。广义上来讲，一个线程只要不是内核线程，都可以认为是用户线程（User Thread，UT）的一种，因此从这个定义上看，轻量级进程也属于用户线程，但轻量级进程的实现始终是建立在内核之上的，许多操作都要进行系统调用，因此效率会受到限制，并不具备通常意义上的用户线程的优点。

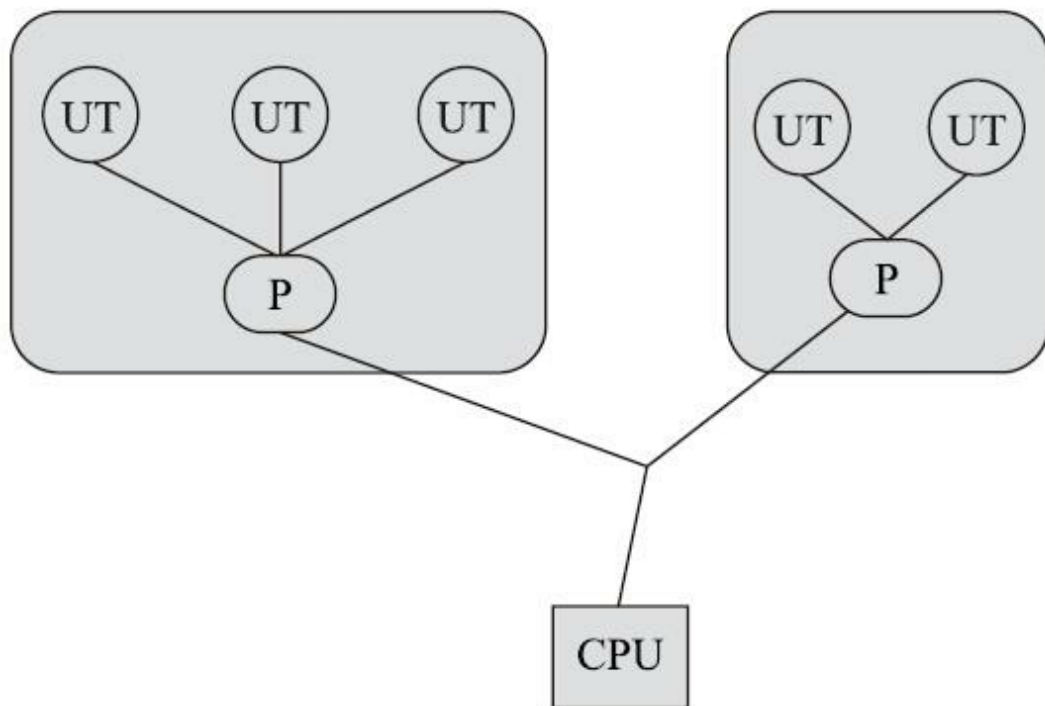


图12-4 进程与用户线程之间1：N的关系

而狭义上的用户线程指的是完全建立在用户空间的线程库上，系统内核不能感知到用户线程的存在及如何实现的。用户线程的建立、同步、销毁和调度完全在用户态中完成，不需要内核的帮助。如果程序实现得当，这种线程不需要切换到内核态，因此操作可以是非常快速且低消耗的，也能够支持规模更大的线程数量，部分高性能数据库中的多线程就是由用户线程实现的。这种进程与用户线程之间1：N的关系称为一对多的线程模型，如图12-4所示。

用户线程的优势在于不需要系统内核支援，劣势也在于没有系统内核的支援，所有的线程操作都需要由用户程序自己去处理。线程的创建、销毁、切换和调度都是用户必须考虑的问题，而且由于操作系统只把处理器资源分配到进程，那诸如“阻塞如何处理”“多处理器系统中如何将线程映射到其他处理器上”这类问题解决起来将会异常困难，甚至有些是不可能实现的。因为使用用户线程实现的程序通常都比较复杂^[1]，除了有明确的需求外（譬如以前在不支持多线程的操作系统中的多线程程序、需要支持大规模线程数量的应用），一般的应用程序都不倾向使用用户线程。Java、Ruby等语言都曾经使用过用户线程，最终又都放弃了使用它。但是近年来许多新的、以高并发为卖点的编程语言又普遍支持了用户线程，譬如Golang、Erlang等，使得用户线程的使用率有所回升。

3.混合实现

线程除了依赖内核线程实现和完全由用户程序自己实现之外，还有一种将内核线程与用户线程一

起使用的实现方式，被称为N：M实现。在这种混合实现下，既存在用户线程，也存在轻量级进程。用户线程还是完全建立在用户空间中，因此用户线程的创建、切换、析构等操作依然廉价，并且可以支持大规模的用户线程并发。而操作系统支持的轻量级进程则作为用户线程和内核线程之间的桥梁，这样可以使用内核提供的线程调度功能及处理器映射，并且用户线程的系统调用要通过轻量级进程来完成，这大大降低了整个进程被完全阻塞的风险。在这种混合模式中，用户线程与轻量级进程的数量比是不定的，是N：M的关系，如图12-5所示，这种就是多对多的线程模型。

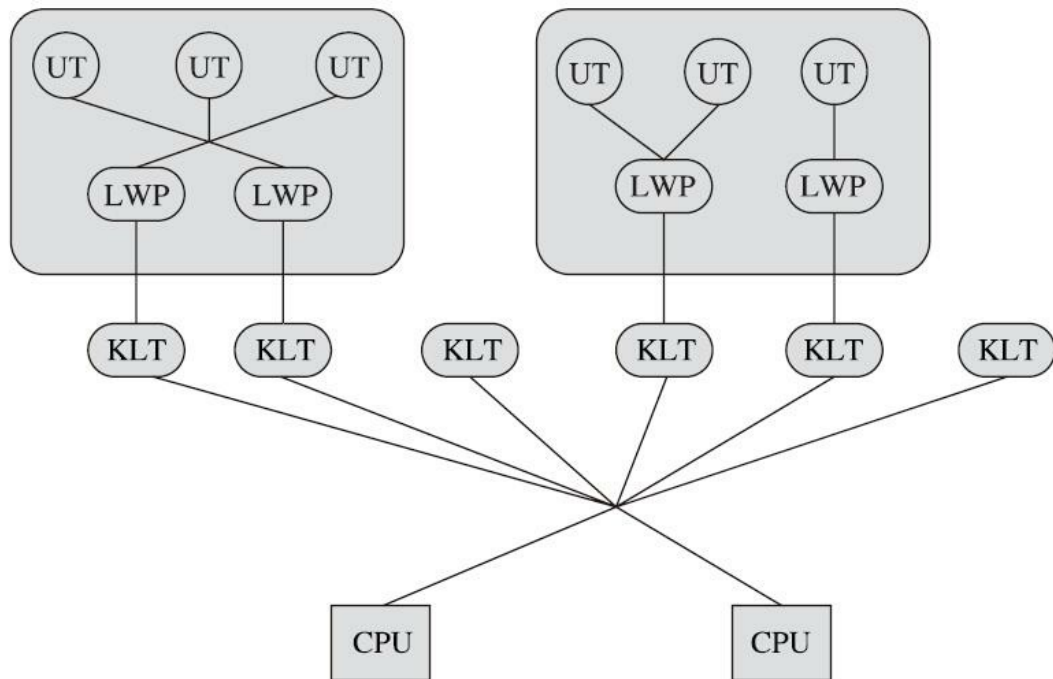


图12-5 用户线程与轻量级进程之间M：N的关系

许多UNIX系列的操作系统，如Solaris、HP-UX等都提供了M：N的线程模型实现。在这些操作系统上的应用也相对更容易应用M：N的线程模型。

4.Java线程的实现

Java线程如何实现并不受Java虚拟机规范的约束，这是一个与具体虚拟机相关的话题。Java线程在早期的Classic虚拟机上（JDK 1.2以前），是基于一种被称为“绿色线程”（Green Threads）的用户线程实现的，但从JDK 1.3起，“主流”平台上的“主流”商用Java虚拟机的线程模型普遍都被替换为基于操作系统原生线程模型来实现，即采用1：1的线程模型。

以HotSpot为例，它的每一个Java线程都是直接映射到一个操作系统原生线程来实现的，而且中间没有额外的间接结构，所以HotSpot自己是不去干涉线程调度的（可以设置线程优先级给操作系统提

供调度建议），全权交给底下的操作系统去处理，所以何时冻结或唤醒线程、该给线程分配多少处理器执行时间、该把线程安排给哪个处理器核心去执行等，都是由操作系统完成的，也都是由操作系统全权决定的。

前面强调是两个“主流”，那就说明肯定还有例外的情况，这里举两个比较著名的例子，一个是用Java ME的CLDC HotSpot Implementation（CLDC-HI，介绍可见第1章）。它同时支持两种线程模型，默认使用1：N由用户线程实现的线程模型，所有Java线程都映射到一个内核线程上；不过它也可以使用另一种特殊的混合模型，Java线程仍然全部映射到一个内核线程上，但当Java线程要执行一个阻塞调用时，CLDC-HI会为该调用单独开一个内核线程，并且调度执行其他Java线程，等到那个阻塞调用完成之后再重新调度之前的Java线程继续执行。

另外一个例子是在Solaris平台的HotSpot虚拟机，由于操作系统的线程特性本来就可以同时支持1：1（通过Bound Threads或Alternate Libthread实现）及N：M（通过LWP/Thread Based Synchronization实现）的线程模型，因此Solaris版的HotSpot也对应提供了两个平台专有的虚拟机参数，即-XX：+UseLWPSynchronization（默认值）和-XX：+UseBoundThreads来明确指定虚拟机使用哪种线程模型。

操作系统支持怎样的线程模型，在很大程度上会影响上面的Java虚拟机的线程是怎样映射的，这一点在不同的平台上很难达成一致，因此《Java虚拟机规范》中才不去限定Java线程需要使用哪种线程模型来实现。线程模型只对线程的并发规模和操作成本产生影响，对Java程序的编码和运行过程来说，这些差异都是完全透明的。

[1] 此处所讲的“复杂”与“程序自己完成线程操作”，并不限于程序直接编写了复杂的实现用户线程的代码，使用用户线程的程序时，很多都依赖特定的线程库来完成基本的线程操作，这些复杂性都封装在线程库之中。

12.4.2 Java线程调度

线程调度是指系统为线程分配处理器使用权的过程，调度主要方式有两种，分别是协同式（Cooperative Threads-Scheduling）线程调度和抢占式（Preemptive Threads-Scheduling）线程调度。

如果使用协同式调度的多线程系统，线程的执行时间由线程本身来控制，线程把自己的工作执行完了之后，要主动通知系统切换到另外一个线程上去。协同式多线程的最大好处是实现简单，而且由于线程要把自己的事情干完之后才会进行线程切换，切换操作对线程自己是可知的，所以一般没有什么线程同步的问题。Lua语言中的“协同例程”就是这类实现。它的坏处也很明显：线程执行时间不可控制，甚至如果一个线程的代码编写有问题，一直不告知系统进行线程切换，那么程序就会一直阻塞在那里。很久以前的Windows 3.x系统就是使用协同式来实现多进程多任务的，那是相当不稳定的，只要有一个进程坚持不交出处理器执行时间，就可能会导致整个系统崩溃。

如果使用抢占式调度的多线程系统，那么每个线程将由系统来分配执行时间，线程的切换不由线程本身来决定。譬如在Java中，有Thread::yield()方法可以主动让出执行时间，但是如果想要主动获取执行时间，线程本身是没有什么办法的。在这种实现线程调度的方式下，线程的执行时间是系统可控的，也不会有一个线程导致整个进程甚至整个系统阻塞的问题。Java使用的线程调度方式就是抢占式调度。与前面所说的Windows 3.x的例子相对，在Windows 9x/NT内核中就是使用抢占式来实现多进程的，当一个进程出了问题，我们还可以使用任务管理器把这个进程杀掉，而不至于导致系统崩溃。

虽然说Java线程调度是系统自动完成的，但是我们仍然可以“建议”操作系统给某些线程多分配一点执行时间，另外的一些线程则可以少分配一点——这项操作是通过设置线程优先级来完成的。Java语言一共设置了10个级别的线程优先级（Thread.MIN_PRIORITY至Thread.MAX_PRIORITY）。在两个线程同时处于Ready状态时，优先级越高的线程越容易被系统选择执行。

不过，线程优先级并不是一项稳定的调节手段，很显然因为主流虚拟机上的Java线程是被映射到系统的原生线程上来实现的，所以线程调度最终还是由操作系统说了算。尽管现代的操作系统基本都提供线程优先级的概念，但是并不见得能与Java线程的优先级一一对应，如Solaris中线程有2147483648（2的31次幂）种优先级，但Windows中就只有七种优先级。如果操作系统的优先级比Java线程优先级更多，那问题还比较好处理，中间留出一点空位就是了，但对于比Java线程优先级少的系统，就不得不出现几个线程优先级对应到同一个操作系统优先级的情况了。表12-1显示了Java线程优先级与Windows线程优先级之间的对应关系，Windows平台的虚拟机中使用了除THREAD_PRIORITY_IDLE之外的其余6种线程优先级，因此在Windows下设置线程优先级为1和2、3和4、6和7、8和9的效果是完全相同的。

表12-1 Java线程优先级与Windows线程优先级之间的对应关系

Java 线程优先级	Windows 线程优先级
1 (Thread.MIN_PRIORITY)	THREAD_PRIORITY_LOWEST
2	THREAD_PRIORITY_LOWEST
3	THREAD_PRIORITY_BELOW_NORMAL
4	THREAD_PRIORITY_BELOW_NORMAL

(续)

Java 线程优先级	Windows 线程优先级
5 (Thread.NORM_PRIORITY)	THREAD_PRIORITY_NORMAL
6	THREAD_PRIORITY_ABOVE_NORMAL
7	THREAD_PRIORITY_ABOVE_NORMAL
8	THREAD_PRIORITY_HIGHEST
9	THREAD_PRIORITY_HIGHEST
10 (Thread.MAX_PRIORITY)	THREAD_PRIORITY_CRITICAL

线程优先级并不是一项稳定的调节手段，这不仅仅体现在某些操作系统上不同的优先级实际会变得相同这一点上，还有其他情况让我们不能过于依赖线程优先级：优先级可能会被系统自行改变，例如在Windows系统中存在一个叫“优先级推进器”的功能（Priority Boosting，当然它可以被关掉），大致作用是当系统发现一个线程被执行得特别频繁时，可能会越过线程优先级去为它分配执行时间，从而减少因为线程频繁切换而带来的性能损耗。因此，我们并不能在程序中通过优先级来完全准确判断一组状态都为Ready的线程将会先执行哪一个。

12.4.3 状态转换

Java语言定义了6种线程状态，在任意一个时间点中，一个线程只能有且只有其中的一种状态，并且可以通过特定的方法在不同状态之间转换。这6种状态分别是：

- 新建（New）：创建后尚未启动的线程处于这种状态。

- 运行（Runnable）：包括操作系统线程状态中的Running和Ready，也就是处于此状态的线程有可能正在执行，也有可能正在等待着操作系统为它分配执行时间。

- 无限期等待（Waiting）：处于这种状态的线程不会被分配处理器执行时间，它们要等待被其他线程显式唤醒。以下方法会让线程陷入无限期的等待状态：

- 没有设置Timeout参数的Object::wait()方法；

- 没有设置Timeout参数的Thread::join()方法；

- LockSupport::park()方法。

- 限期等待（Timed Waiting）：处于这种状态的线程也不会被分配处理器执行时间，不过无须等待被其他线程显式唤醒，在一定时间之后它们会由系统自动唤醒。以下方法会让线程进入限期等待状态：

- Thread::sleep()方法；

- 设置了Timeout参数的Object::wait()方法；

- 设置了Timeout参数的Thread::join()方法；

- LockSupport::parkNanos()方法；

- LockSupport::parkUntil()方法。

- 阻塞（Blocked）：线程被阻塞了，“阻塞状态”与“等待状态”的区别是“阻塞状态”在等待着获取到一个排它锁，这个事件将在另外一个线程放弃这个锁的时候发生；而“等待状态”则是在等待一段时间，或者唤醒动作的发生。在程序等待进入同步区域的时候，线程将进入这种状态。

- 结束（Terminated）：已终止线程的线程状态，线程已经结束执行。

上述6种状态在遇到特定事件发生的时候将会互相转换，它们的转换关系如图12-6所示。

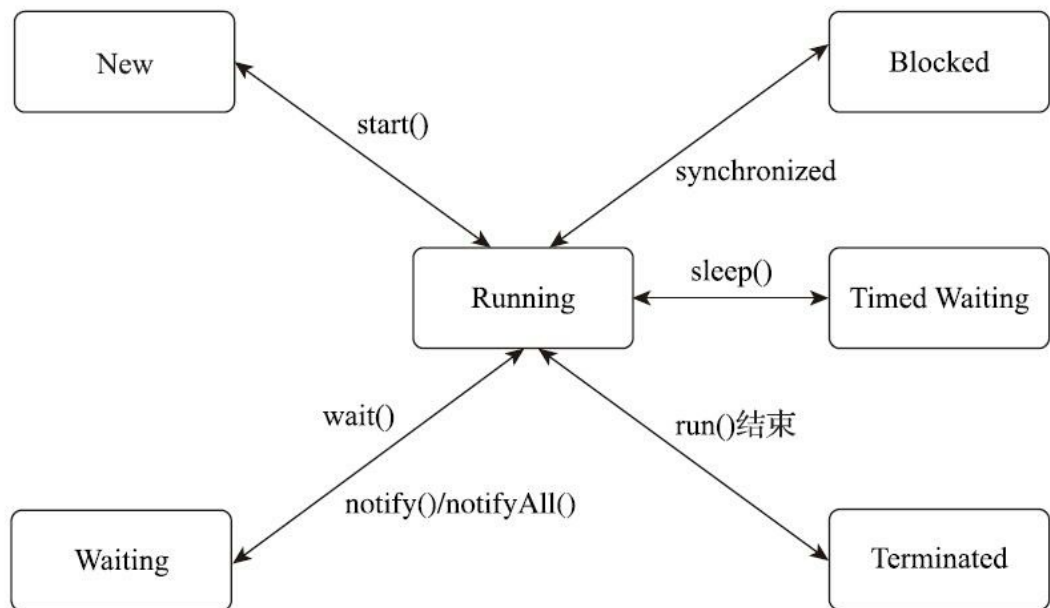


图12-6 线程状态转换关系

12.5 Java与协程

在Java时代的早期，Java语言抽象出来隐藏了各种操作系统线程差异性的统一线程接口，这曾经是它区别于其他编程语言的一大优势。在此基础上，涌现过无数多线程的应用与框架，譬如在网页访问时，HTTP请求可以直接与Servlet API中的一条处理线程绑定在一起，以“一对一服务”的方式处理由浏览器发来的信息。语言与框架已经自动屏蔽了相当多同步和并发的复杂性，对于普通开发者而言，几乎不需要专门针对多线程进行学习训练就能完成一般的并发任务。时至今日，这种便捷的并发编程方式和同步的机制依然在有效地运作着，但是在某些场景下，却也已经显现出了疲态。

12.5.1 内核线程的局限

笔者可以通过一个具体场景来解释目前Java线程面临的困境。今天对Web应用的服务要求，不论是在请求数量上还是在复杂度上，与十多年前相比已不可同日而语，这一方面是源于业务量的增长，另一方面来自于为了应对业务复杂化而不断进行的服务细分。现代B/S系统中一次对外部业务请求的响应，往往需要分布在不同机器上的大量服务共同协作来实现，这种服务细分的架构在减少单个服务复杂度、增加复用性的同时，也不可避免地增加了服务的数量，缩短了留给每个服务的响应时间。这要求每一个服务都必须在极短的时间内完成计算，这样组合多个服务的总耗时才不会太长；也要求每一个服务提供者都要能同时处理数量更庞大的请求，这样才不会出现请求由于某个服务被阻塞而出现等待。

Java目前的并发编程机制就与上述架构趋势产生了一些矛盾，1：1的内核线程模型是如今Java虚拟机线程实现的主流选择，但是这种映射到操作系统上的线程天然的缺陷是切换、调度成本高昂，系统能容纳的线程数量也很有限。以前处理一个请求可以允许花费很长时间在单体应用中，具有这种线程切换的成本也是无伤大雅的，但现在在每个请求本身的执行时间变得很短、数量变得很多的前提下，用户线程切换的开销甚至可能会接近用于计算本身的开销，这就会造成严重的浪费。

传统的Java Web服务器的线程池的容量通常在几十个到两百之间，当程序员把数以百万计的请求往线程池里面灌时，系统即使能处理得过来，但其中的切换损耗也是相当可观的。现实的需求在迫使Java去研究新的解决方案，同大家又开始怀念以前绿色线程的种种好处，绿色线程已随着Classic虚拟机的消失而被尘封到历史之中，它还会有重现天日的一天吗？

12.5.2 协程的复苏

经过前面对不同线程实现方式的铺垫介绍，我们已经明白了各种线程实现方式的优缺点，所以多数读者看到笔者写“因为映射到了系统的内核线程中，所以切换调度成本会比较高昂”时并不会觉得有什么问题，但相信还是有一部分治学特别严谨的读者会提问：为什么内核线程调度切换起来成本就要更高？

内核线程的调度成本主要来自于用户态与核心态之间的状态转换，而这两种状态转换的开销主要来自于响应中断、保护和恢复执行现场的成本。请读者试想以下场景，假设发生了这样一次线程切换：

线程A -> 系统中断 -> 线程B

处理器要去执行线程A的程序代码时，并不是仅有代码程序就能跑得起来，程序是数据与代码的组合物，代码执行时还必须要上下文数据的支撑。而这里说的“上下文”，以程序员的角度来看，是方法调用过程中的各种局部的变量与资源；以线程的角度来看，是方法的调用栈中存储的各类信息；而以操作系统和硬件的角度来看，则是存储在内存、缓存和寄存器中的一个具体数值。物理硬件的各种存储设备和寄存器是被操作系统内所有线程共享的资源，当中断发生，从线程A切换到线程B去执行之前，操作系统首先要把线程A的上下文数据妥善保管好，然后把寄存器、内存分页等恢复到线程B挂起时候的状态，这样线程B被重新激活后才能仿佛从来没有被挂起过。这种保护和恢复现场的工作，免不了涉及一系列数据在各种寄存器、缓存中的来回拷贝，当然不可能是一种轻量级的操作。

如果说内核线程的切换开销是来自于保护和恢复现场的成本，那如果改为采用用户线程，这部分开销就能够省略掉吗？答案是“不能”。但是，一旦把保护、恢复现场及调度的工作从操作系统交到程序员手上，那我们就可以打开脑洞，通过玩出很多新的花样来缩减这些开销。

有一些古老的操作系统（譬如DOS）是单人单工作业形式的，天生就不支持多线程，自然也不会有多个调用栈这样的基础设施。而早在那样的蛮荒时代，就已经出现了今天被称为栈纠缠（Stack Twine）的、由用户自己模拟多线程、自己保护恢复现场的工作模式。其大致的原理是通过在内存里划出一片额外空间来模拟调用栈，只要其他“线程”中方法压栈、退栈时遵守规则，不破坏这片空间即可，这样多段代码执行时就会像相互缠绕着一样，非常形象。

到后来，操作系统开始提供多线程的支持，靠应用自己模拟多线程的做法自然是变少了许多，但也并没有完全消失，而是演化为用户线程继续存在。由于最初多数的用户线程是被设计成协同式调度（Cooperative Scheduling）的，所以它有了一个别名——“协程”（Coroutine）。又由于这时候的协程会完整地做调用栈的保护、恢复工作，所以今天也被称为“有栈协程”（Stackfull Coroutine），起这样的名字是为了便于跟后来的“无栈协程”（Stackless Coroutine）区分开。无栈协程不是本节的主角，不过还是可以简单提一下它的典型应用，即各种语言中的await、async、yield这类关键字。无栈协程本质上是一种有限状态机，状态保存在闭包里，自然比有栈协程恢复调用栈要轻量得多，但功能也相对更有限。

协程的主要优势是轻量，无论是有栈协程还是无栈协程，都要比传统内核线程要轻量得多。如果

进行量化的话，那么如果不显式设置-Xss或-XX: ThreadStackSize，则在64位Linux上HotSpot的线程栈容量默认是1MB，此外内核数据结构（Kernel Data Structures）还会额外消耗16KB内存。与之相对的，一个协程的栈通常在几百个字节到几KB之间，所以Java虚拟机里线程池容量达到两百就已经不算小了，而很多支持协程的应用中，同时并存的协程数量可数以十万计。

协程当然也有它的局限，需要在应用层面实现的内容（调用栈、调度器这些）特别多，这个缺点就不赘述了。除此之外，协程在最初，甚至在今天很多语言和框架中会被设计成协同式调度，这样在语言运行平台或者框架上的调度器就可以做得非常简单。不过有不少资料上显示，既然取了“协程”这样的名字，它们之间就一定以协同调度的方式工作。笔者并没有查证到这种“规定”的出处，只能说这种提法在今天太过狭隘了，非协同式、可自定义调度的协程的例子并不少见，而协同调度的优点与不足在12.4.2节已经介绍过。

具体到Java语言，还会有一些别的限制，譬如HotSpot这样的虚拟机，Java调用栈跟本地调用栈是做在一起的。如果在协程中调用了本地方法，还能否正常切换协程而不影响整个线程？另外，如果协程中遇传统的线程同步措施会怎样？譬如Kotlin提供的协程实现，一旦遭遇synchronize关键字，那挂起来的仍将是整个线程。

12.5.3 Java的解决方案

对于有栈协程，有一种特例实现名为纤程（Fiber），这个词最早是来自微软公司，后来微软还推出过系统层面的纤程包来方便应用做现场保存、恢复和纤程调度。OpenJDK在2018年创建了Loom项目，这是Java用来应对本节开篇所列场景的官方解决方案，根据目前公开的信息，如无意外，日后该项目为Java语言引入的、与现在线程模型平行的新并发编程机制中应该也会采用“纤程”这个名字，不过这显然跟微软是没有任何关系的。从Oracle官方对“什么是纤程”的解释里可以看出，它就是一种典型的有栈协程，如图12-11所示。

What is a fiber?

- A *light weight* or *user mode thread*, scheduled by the Java virtual machine, not the operating system
- Fibers are low footprint and have negligible task-switching overhead. You can have millions of them!



图12-7 JVMLS 2018大会上Oracle对纤程的介绍

Loom项目背后的意图是重新提供对用户线程的支持，但与过去的绿色线程不同，这些新功能不是为了取代当前基于操作系统的线程实现，而是会有两个并发编程模型在Java虚拟机中并存，可以在程序中同时使用。新模型有意地保持了与目前线程模型相似的API设计，它们甚至可以拥有一个共同的基类，这样现有的代码就不需要为了使用纤程而进行过多改动，甚至不需要知道背后采用了哪个并发编程模型。Loom团队在JVMLS 2018大会上公布了他们对Jetty基于纤程改造后的测试结果，同样在5000QPS的压力下，以容量为400的线程池的传统模式和每个请求配以一个纤程的新并发处理模式进行对比，前者的请求响应延迟在10000至20000毫秒之间，而后的延迟普遍在200毫秒以下，具体结果如图12-8所示。

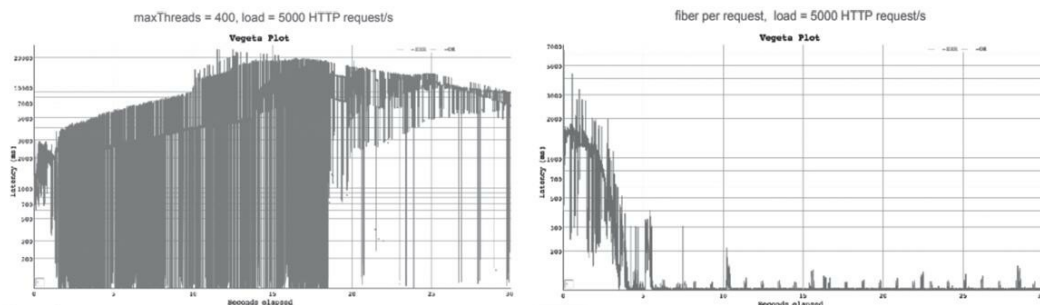


图12-8 Jetty在新并发模型下的压力测试

在新并发模型下，一段使用纤程并发的代码会被分为两部分——执行过程（Continuation）和调度器（Scheduler）。执行过程主要用于维护执行现场，保护、恢复上下文状态，而调度器则负责编排所有要执行的代码的顺序。将调度程序与执行过程分离的好处是，用户可以选择自行控制其中的一个或者多个，而且Java中现有的调度器也可以被直接重用。事实上，Loom中默认的调度器就是原来已存在的用于任务分解的Fork/Join池（JDK 7中加入的ForkJoinPool）。

Loom项目目前仍然在进行当中，还没有明确的发布日期，上面笔者介绍的内容日后都有被改动的可能。如果读者现在就想尝试协程，那可以在项目中使用Quasar协程库^[1]，这是一个不依赖Java虚拟机的独立实现的协程库。不依赖虚拟机来实现协程是完全可能的，Kotlin语言的协程就已经证明了这一点。Quasar的实现原理是字节码注入，在字节码层面对当前被调用函数中的所有局部变量进行保存和恢复。这种不依赖Java虚拟机的现场保护虽然能够工作，但很影响性能，对即时编译器的干扰也非常大，而且必须要求用户手动标注每一个函数是否会在协程上下文被调用，这些都是未来Loom项目要解决的问题。

[1] 如同JDK 5把Doug Lea的dl.util.concurrent项目引入，成为java.util.concurrent包，JDK 9时把Attila Szegedi的dynamalink项目引入，成为jdk.dynamalink模块。Loom项目的领导者Ron Pressler就是Quasar的作者。